MyCodeSchool

Code review — react-initiere

Rareș · runde de review pe proiectul tău, cea mai nouă sus.

Runda 1 — 2026-10-27 — sha cfa08ad

Ai rezolvat exercițiile 1a–7a (31 din 50). Proiectul se compilează (`npm run build` ✅), iar fiecare exercițiu l-am rulat în browser și l-am comparat cu „Pe ecran" din fișă. Tipurile sunt puse peste tot, `useState<T>` e folosit corect, iar `map` cu `key` e bine din prima. Rămân 2 bug-uri reale și câteva lucruri mici.

🔴 Critice

B1 🔴 Nota 5 iese „Respins" `src/exercitii/Ex6dGrade.tsx:14`

CUM E ACUM: scrii 5 în input → pe ecran apare **Respins**. Fișa cere „de la 5 în sus Admis".

DE CE: `>` și `>=` diferă exact pe valoarea de la graniță. „De la 5 în sus" include 5, iar `grade > 5` îl lasă afară. Bug-ul ăsta nu se vede dacă testezi cu 4 și 6: amândouă dau răspunsul corect. Se vede doar dacă încerci chiar valoarea din enunț. La orice condiție cu prag testezi trei valori: una sub, **una pe** și una peste.

FIX: `grade >= 5`

B2 🔴 Anul nașterii acceptă litere și afișează `NaN` `src/exercitii/Ex5dBirthYear.tsx:9–11`

CUM E ACUM: scrii `200a` → pe ecran apare **În 2026 ai NaN ani.**, iar în input rămâne textul `NaN`.

DE CE: un `<input>` fără `type` e un câmp de text și primește orice tastezi. `Number("200a")` nu dă eroare, ci întoarce `NaN` („Not a Number"), care e tot o valoare de tip `number`. De aceea TypeScript nu se plânge. `NaN` intră în stare, iar orice calcul cu el dă tot `NaN`: `2026 - NaN` este `NaN`. Cu `type="number"`, browserul nu mai lasă literele să intre în câmp, deci `NaN` nu mai ajunge în stare. Fișa cere `type="number"` exact din cauza asta. La 6c și 6d l-ai pus, aici nu.

FIX: `<input type="number" ...>`

🟡 Importante

- **M1** `Ex6cEvenOdd.tsx:15,17` : pe ecran apare `"Par"` **cu ghilimele**. În JSX, textul dintre taguri se scrie fără ghilimele: `<p>Par</p>`. Ghilimelele se pun doar în `{ }`, de exemplu `{"Par"}`.
- **M2** `Ex6eLogin.tsx:18` : `error` devine `true` la prima greșeală și nu mai revine la `false`. Mesajul roșu rămâne pe ecran și după ce începi să scrii din nou. Pune `setError(false)` la începutul lui `login()`.
- **M3** `==` în loc de `===` (`Ex6cEvenOdd.tsx:14`, `Ex6bSecret.tsx:10`). `==` convertește tipurile înainte să compare: `"0" == 0` dă `true`. În TypeScript folosești mereu `===`. La 6b nici nu-ți trebuie comparația: `visible ? "Ascunde" : "Arată"`.

🟢 Mărunte

- **C1** `Ex5bFullName.tsx:10–17` : `<input ...></input>` → `<input ... />`. `input` nu are conținut (de aici cele 2 warning-uri din `oxlint`). La fel în 5c, 5d și 5e.
- **C2** `Ex4dStep.tsx:5` : `Setcount` → `setCount`. Setter-ul se scrie cu `set` mic, ca toate celelalte. Tot acolo, `onClick={(reset)}` → `onClick={reset}` și `() => (add(1))` → `() => add(1)`.
- **C3** `Ex2cTemperature.tsx:9` : `25°C` e scris de mână. Dacă schimbi `celsius`, textul rămâne 25. Scrie `{celsius}°C`.
- **C4** `Ex4bVotes.tsx:10` : `;` în plus după acolada funcției.
- **C5** `Ex5aCharCount.tsx:14,17` : `<h1>Salut!</h1>` și butonul „Șterge" nu sunt cerute. Nu strică nimic, dar exercițiul cere doar ce scrie în fișă.

Before / After

	Acum	Corect
B1	`{ grade > 5 ? (`	`{ grade >= 5 ? (`
B2	`<input value={year} onChange={...}></input>`	`<input type="number" value={year} onChange={...} />`

Întrebări

1. La 4a ai scris `if (count > 0)` și e corect. De ce acolo `>` e bun, iar la 6d nu?
2. Ce afișează `{2026 - Number("")}`? Încearcă la 5d, după fix: șterge tot din input. E ce te-ai așteptat?
3. La 6e: ce vezi pe ecran dacă greșești parola o dată, apoi scrii `admin` / `1234` și apeși „Intră"? Dar dacă greșești de două ori?

Următorul pas: 7b–7e (lista în stare: ștergere și adăugare), apoi demo-urile 8–10.